Programming Languages and Techniques (CIS1200)

Lecture 9

Property-based testing

Higher-order functions: transform and fold

Lecture notes: Chapter 9

Announcements (1)

- Homework 3 available this evening
  - Practice with BSTs, property-based testing, generic functions, first-class functions, abstract types
  - Due Date:
    - Tuesday, Feb 10th at 11:59pm*
  - *Start early!*
    - *Problems 0-3 (and part of 4) can be done after class today*
    - *Problems 4-8 can be done after class on Friday*

- Reading: Chapters 8, 9, and 10 of the lecture notes

Property-Based Testing

A little recursive function

```
let rec take (n : int) (xs : int list) : int list =
  if n <= 0 then []
  else begin match xs with
             | [] -> []
             | x :: xs' -> x :: take (n - 1) xs'
       end
```

```
take 2 [1;2;3;4]  =  [1;2]
take 0 [1;2;3;4]  =  []
take 6 [1;2;3;4]  =  [1;2;3;4]
```

Some unit tests

```
let test () : bool =
  take 0 [1;2;3] = []
;; run_test "..." test

let test () : bool =
  take 0 [] = []
;; run_test "..." test
```

```
let rec take (n : int) (xs : int list) : int list =
  if n <= 0 then []
  else begin match xs with
             | [] -> []
             | x :: xs' -> x :: take (n - 1) xs'
       end
```

A common pattern

```
let test () : bool =
  take 0 [1;2;3] = []
;; run_test "..." test

let test () : bool =
  take 0 [] = []
;; run_test "..." test
```

Seems like applying `take 0` to *any* list yields an empty list…

```
let rec take (n : int) (xs : int list) : int list =
  if n <= 0 then []
  else begin match xs with
             | [] -> []
             | x :: xs' -> x :: take (n - 1) xs'
       end
```

Generalize the pattern

```
let property (xs : int list) : bool =
  take 0 xs = []

let test () : bool = property [1;2;3]
;; run_test "..." test

let test () : bool = property []
;; run_test "..." test
```

Define a generic **property**

Instantiate it for multiple tests

```
let rec take (n : int) (xs : int list) : int list =
  if n <= 0 then []
  else begin match xs with
            | [] -> []
            | x :: xs' -> x :: take (n - 1) xs'
       end
```

Why was this step useful?

1. We've extracted the common pattern from multiple unit tests into a general **property** of the function.
   1. Helps us think about / document what the function **does**
   2. More concise than previous unit tests

      → Makes it easy to write *more* unit tests

2. Since each test case now just consists of single list, we can generate a lot of test cases **randomly**!

Generators

```
;; print_samples
     gen_int
     5
     string_of_int
```

A **generator** for random ints

```
Generated: 36
Generated: 22
Generated: 3
Generated: 38
Generated: 98
```

Generators

A generator for random **lists of ints**

```
;; print_samples
     (gen_list gen_int)
     5
     (string_of_list string_of_int)
```

```
  Generated: [6; -9; 8; 8]
  Generated: [-7; -6; -7; 25; -1; -3; 8; 9; 8; 35; 3; 4; -88; 94; 71; 9; -7; 5; 20; -4; -9; 15; -4; 0; 78; -4; -16; -29; 9; -18; 0; 9; 6;
16; 1; -7; -3; 83; -41; 2; 94; 0; -5; 5; 0; 1; -4; 7; 8; 7; -7; -1; -9; -8; -6; 6; -49; -5; 4; -28; -9; 7; 0; 0; -2; 2; 7; -7; 1; 0; 9; -7;
-5; 9; -33; -9; 0; -3; 9; -6; 54; -3; 44; 2; -1; -8; 61; -13; -9; -1; 3; -5; 81; 5; -5; -4; 2; 8; 7; -5; -3; 5; 3; 4; 98; 2; -9; 72; 8; -8;
-7; -8; -64; 4; -1; 9; -4; -58; -5; 88; 2; 1; -5; -4; -9; -5; 4; 8; -3; 0; 2; -77; 0; 8; 0; -7; 4; 92; -47; -6; 1; 0; -74; -5; 2; -3; -1; -
7; 4; -76; 0; -97; 9; -5; -3; -1; 2; -5; 1; -22; 6; -1; 8; 1; -2; -4; 6; -2; 0; 3; 0; 20; -2; 1; 4; -38; 5; 9; 40; -6; 6; -1; -25; -8; 6;
9; -8; -5; 1; -7; 9; 4; 8; 9; -1; 0; 4; 3; 3; 19; -4; 0; 4; 0; -4; 2; 8; -2; -98; 0; -9; -8; -3; -2; 2; 7; -72; -2; -2; 3; 0; 0; 8; -8; 0;
0; -3; 3; -5; -6; -5; -3; -6; -4; -2; -90; -9; 69; -36; -35; -8; 7; 0; -5; 6; -8; -7; -4; -5; -3; -8; -57; -2; -4; 5; 7; -7; 4; 65; -66; 2;
-2; 69; 3; 57; 2; 1; -3; 9; 3; 5; 4; 69; 0; 3; -2; 1; -3; 9; -51; 66; 8; -8; 2; -4; -67; -6; -73; 6; -6; 7; -4; -4; 41; 1; -6; 44; -6; 6;
8; -7; -8; 0; -1; -1; 4; -60; -5; 3; 4; -9; 5; -99; -2; -73; -5; 50; 0; -4; -7; -9; -54; -9; 2; 1; 7; 92; -5; 8; 6; -7; 6; -40; 6; 75; 0; -
1; 19; -8; 0; 0; -6; 1; -33; 0; 2; 2; -7; -4; 22; 4; -1; 0; 48; 86; 0; 7; -51; 5; -6; 44; -2; -6; -25; 2; -4; 2; -28; 86; -8; 8; 7; -8; 2;
5; 2; -7; -2; -3; -52; 0; -6; 3; -6; 94; 4; 5; 8; 3; -82; 4; 2; -3; 9; -8; 9; -3; -1; -7; -8; -6; -12; -6; -1; -3; 9; -7; 4; -1; -6; 93; 7;
-9; -4; -1; -2; 0; -11; -6; -9; -1; -3; -1; -56; 12; -7; -7; -3; 3; 6; 4; -8; 9; 2; -5; 9; 1; 5; -6; 77; -30; 8; 31; 6; -82; 9; 63; 6; -8;
5; 7; 74; -64; 62; 8; -4; -8; 8; -60; 0; 98; 84; 0; 7; -1; 84; -2; -2; -2; 3; -8; 0; 92; 29; 6; -4; 8; -9; -23; -2; 7; -9; -4; 9; 8; 10; 0;
-95; 8; -5; 9; 4; -8; -2; -9; 8; 94; 0; 4; 0; -3; -9; 5; 7; 0; 21; 1; 83; 6; -3; -6; -3; 3; -14; 3; 6; 11; 9; 2; -53; 72; -55; -9; 0; 0; -
5; 31; -5; 6; -7; 7; 34; 53; -2; 1; -5; -5; 78; -6; -4; -70; 0; 48; 7; -7; 4; -55; -7; 0; 0; 2; -4; 0; 5; -6; 7; -2; -2; -1; 9; 7; 3; 9;
94; 7; 58; -38; 84; 9; 31; 1; -2; 30; 5; 28; 0; 28]
  Generated: [9; -8; 4; 4; 0]
  Generated: [7; -8; -5; -3; -3; 0; -5; 8; 48; 9; 84; 7; 3; 0; -6; 43; -7; -84; -7; -6; -5; 6; -7; 8; 2; -9; -9; 11; -5]
  Generated: [-6; 8; 0; 89; 8; -2; 6]
```

Putting them together

```
let property (xs : int list) : bool =
  take 0 xs = []

;; run_property_test
     "take 0 xs = []"
     (gen_list gen_int)
     property
```

```
PBT: take 0 xs = []

random seed: 11378409
generated error fail pass / total     time test name
[✓]  100    0    0  100 /  100     0.0s take 0 xs = []
================================================================
success (ran 1 tests)

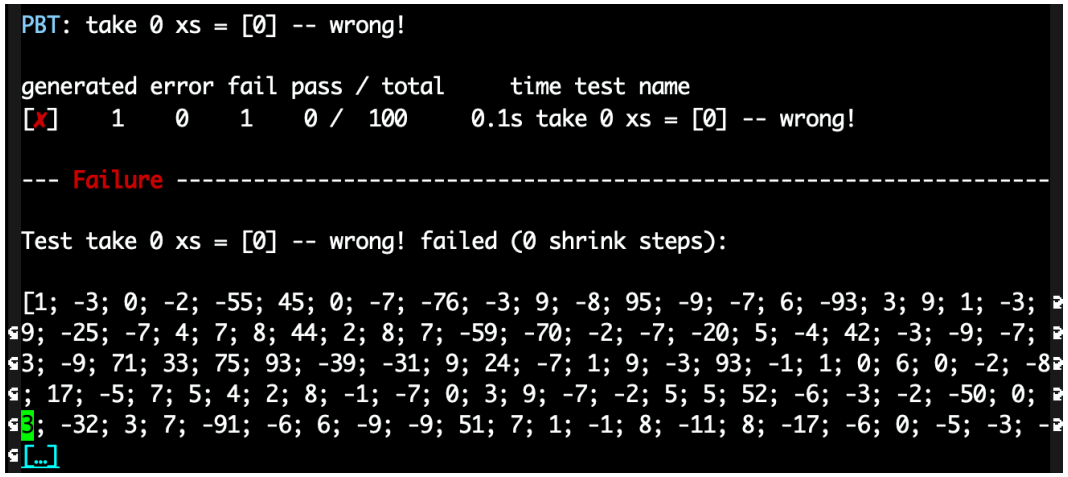
  --- take 0 xs = []: PASSED ---
```

A bad property

```
let wrong_property (xs : int list) : bool =
  take 0 xs = [0]

;; run_property_test
     "take 0 xs = [0] -- wrong!"
     (gen_list_no_shrinker gen_int_no_shrinker)
     wrong_property
```

An unreadable failing case

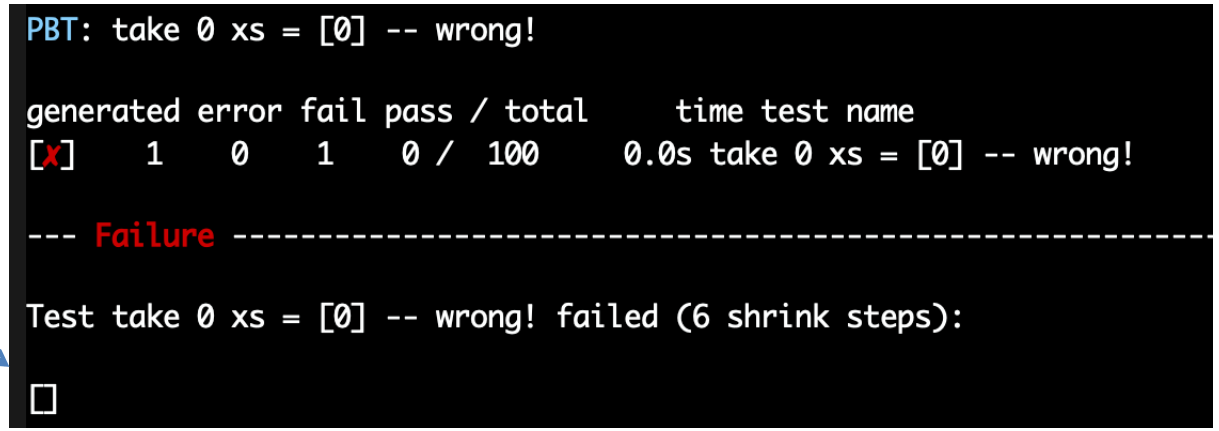


Shrinking

```
let wrong_property (xs : int list) : bool =
  take 0 xs = [0]

;; run_property_test
     "take 0 xs = [0] -- wrong!"
     (gen_list gen_int)
     wrong_property
```

A shrunk failing case



A more interesting test

```
let rec drop (n : int) (xs : int list) : int list =
  if n <= 0 then xs
  else begin match xs with
              | [] -> []
              | _ :: xs' -> drop (n - 1) xs'
       end

let property ((xs, n) : int list * int) : bool =
  take n xs @ drop n xs = xs

;; run_property_test
     "take n @ drop n"
     (gen_pair (gen_list gen_int) gen_int) property
```

Property-Based Testing

PBT = Property + Generator + Shrinking

Is PBT good for everything?

- No
- But it's extremely useful in some common situations…
  - Validity properties
    - Inserting into a BST yields a BST
  - Postcondition properties
    - ”take 0 xs is always empty”
  - Round-trip properties
    - Take/drop
    - Unzip / zip
    - Printing / parsing
  - Combinations of operations
    - Inserting an element into a BST and then deleting it does not change the set of elements

More on PBT in HW03…

Research Study about Property-Based Testing in 1200

Xiaorui Liu

Who am I?

- **Xiaorui Liu**
- **1st year PhD student advised by Andrew Head**
  - Human computer interaction
    - My work is helping programmers through better pedagogy + tooling
  - Work closely with Benjamin as well :)
- **My personal website: xiaoruiyliu.github.io**

What is the study I referred to?

Introduction of Property-Based Testing

- **Integration in…**
  - Lectures
  - Recitations
  - Homeworks 3 and 4
- **Our perspective**
  - Generally, PBT is a valuable skill to learn
  - For assignments, it hopefully will help you get to better solutions!

What are we trying to learn

- **What is it like to learn PBT?**
  - How do you approach using it?
  - How might we/other classes/institutions go about teaching PBT?
- **To learn these things, we need to pay close attention to how the PBT exercises go!**

Study Plans

- **We hope to better understand your experience with PBT this semester**
- **Many parts require no special effort from you**
  - Collecting information about how the PBT parts of the assignments are going
  - Asking questions as parts of the homework

Extra opportunities for engagement

- **Observations of how you complete homeworks**
- **Interviews with how learning and using PBT went**

More information about this on Ed!

We hope to connect with many of you!

- **Many aspects of this study encompass things you would do anyway.**
- **For the optional study sessions, we pay you ($35/hr) for your time**
  - These sessions are not meant to provide extra help for participants
  - Participants typically enjoy getting to share their opinions
  - Opportunity for deeper reflection on the material

What data will we collect?

- **Codio, gradescope submissions, Ed questions**
- **Interview transcripts, observation recordings**

How will we handle and respect your data?

- **All data will be anonymized**
- **Benjamin, me, my advisor, and another senior PhD student will have access to it**
- **Data will be only analyzed for research purposes**
- **Our aim is to share our findings in a research publication**
    - Your individual data will be unrecognizable when we report it

Participation is optional

- **If you don't want us to have access to HW3/HW4 submissions, opt out through this form: http://bit.ly/49Z9YMu**
- **If you're ok with us engaging with more information**
  - Provide us access to anonymized grades here
  - Let us understand the course demographics here
- **If you don't want talk to me/be observed ignore my messages on Ed**
  - I would deeply appreciate your participation!
- **Your choice to participate has absolutely no influence on your standing in the course or relationship with the professor**

Observation sessions are open!

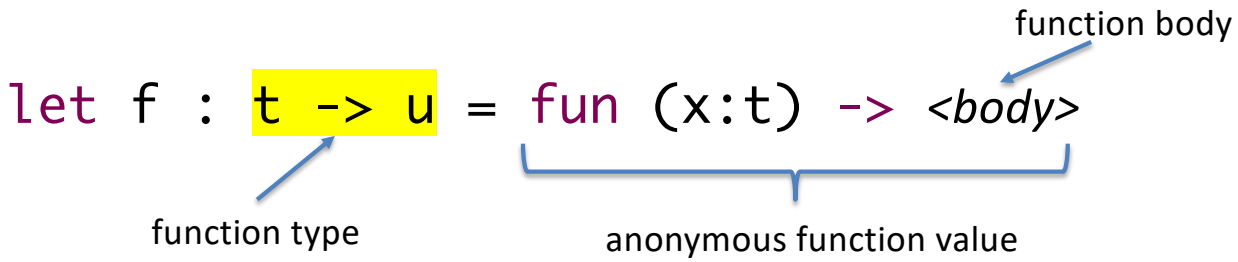
- Very casual 1 hr 15 min study
- Let me watch you complete any PBT sections of HW3
- Answer some questions that come up
- Leave with a $43 virtual debit card!

xrl@seas.upenn.edu/Ed!



First-Class Functions

First-class Functions



- Functions are *first-class values* in OCaml: they can be manipulated like any other value.
- They have a type that specifies the input and output types.
- The "fun" keyword introduces an *anonymous function*.
  - Sometimes called *lambdas** or *closures*

*The term "lambda" comes from Church's *lambda calculus*.

What is the value of this expression?

```
let f (g : int->int) (y: int) : int =
   g 1 + y in

f (fun (x:int) -> x + 1) 3
```

1. 1
2. 2
3. 3
4. 4
5. 5

Answer: 5

Functions can take functions as arguments

```
let f (g : int->int) (y: int) : int =
    (g 1) + y in
f (fun (x:int) -> x + 1) 3
```

⟼ *rewrite let with anonymous function*

```
let f = fun (g : int->int) -> fun (y: int) -> (g 1) + y in
f (fun (x:int) -> x + 1) 3
```

⟼ *substitute for f, drop let*

```
(fun (g : int->int) -> fun (y: int) -> (g 1) + y)
    (fun (x:int) -> x + 1) 3
```

⟼ *substitute for g and y (two steps)*

```
((fun (x:int) -> x + 1) 1) + 3
```

⟼ *substitute for x*

```
(1 + 1) + 3
```

What is the value of this expression?

Functions can return functions

What is the value of this expression?

```
let f (x:bool) (y:int) : int =
    if x then 1 else y in
f true
```

the result of a "partially applied function" is itself a function (that can later be applied)

```
let f (x:bool) (y:int) : int = if x then 1 else y in f true
```

⟼ *rewrite let with anonymous function*

```
let f = fun (x:bool) -> fun (y:int) -> if x then 1 else y in  f true
```

⟼ *substitute for f in body of let, drop let*

```
(fun (x:bool) -> fun (y:int) -> if x then 1 else y) true
```

⟼ *substitute for x in body of function*

```
fun (y:int) -> if true then 1 else y
```

What good is all this?

Consider this *filter* function:

```
let rec filter (p:'a -> bool) (l:'a list) : 'a list =
  begin match l with
  | [] -> []
  | x::xs -> if p x then x :: (filter p xs) else (filter p xs)
  end
```

`filter` selects elements of a list based on a predicate p.

```
let larger  = filter (fun x -> x > 10)
let smaller = filter (fun x -> x <= 10)
```

We can create specialized "instances" by partial application…

```
larger  [1;17;120;4;10] ⇒ [17;120]
smaller [1;17;120;4;10] ⇒ [1;4;10]
```

and use them as ordinary list-processing functions.

Upshot: higher-order functions like filter are a very useful way to structure library interfaces…

List transformations

A fundamental design pattern
using first-class functions

Phone book example

```
type entry = string * int
let phone_book = [ ("Stephanie", 2155559092); … ]

let rec get_names (p : entry list) : string list =
  begin match p with
  | ((name, num)::rest) -> name :: get_names rest
  | [] -> []
  end

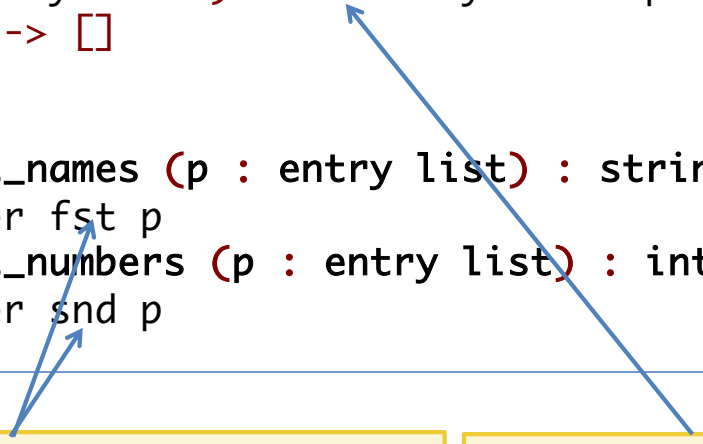
let rec get_numbers (p : entry list) : int list =
  begin match p with
  | ((name, num)::rest) -> num :: get_numbers rest
  | [] -> []
  end
```

Can we use first-class functions to refactor code to share common structure?

Refactoring

```
let rec helper (f:entry -> 'b) (p:entry list) : 'b list =
  begin match p with
  | (entry::rest) -> f entry :: helper f rest
  | [] -> []
  end

let get_names (p : entry list) : string list =
  helper fst p
let get_numbers (p : entry list) : int list =
  helper snd p
```



fst and snd are functions that access the parts of a tuple:

```
let fst (x,y) = x
let snd (x,y) = y
```

The argument `f` determines what happens with the entry at the head of the list

Going even more generic

```
let rec helper (f:entry -> 'b) (p:entry list) : 'b list =
  begin match p with
  | (entry::rest) -> f entry :: helper f rest
  | [] -> []
  end

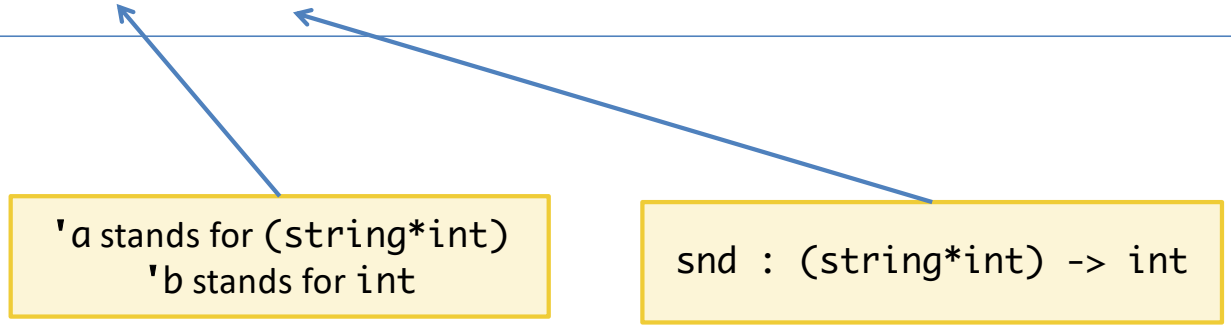
let get_names (p : entry list) : string list =
  helper fst p
let get_numbers (p : entry list) : int list =
  helper snd p
```

Now let's make it work for *all* lists, not just lists of entries…

Going even more generic

```
let rec helper (f:'a -> 'b) (p:'a list) : 'b list =
  begin match p with
  | (entry::rest) -> f entry :: helper f rest
  | [] -> []
  end

let get_names (p : entry list) : string list =
  helper fst p
let get_numbers (p : entry list) : int list =
  helper snd p
```



Transforming Lists

```
let rec transform (f: 'a->'b) (l:'a list) : 'b list =
  begin match l with
  | []   -> []
  | h::t -> (f h)::(transform f t)
  end
```

List *transformation*

a.k.a. "*mapping** a function across a list"

- foundational function for programming with lists
- part of OCaml standard library (called List.map)
- used over and over again

  (e.g., Google's famous *map*-reduce infrastructure)

*Many languages (including OCaml) use the terminology "map" for the function that transforms a list by applying a function to each element. Don't confuse List.map with "finite map".

9: What is the value of this expresssion?

0

```
transform (fun (x:int) -> x > 0)
    [0 ; -1; 1; -2]
```

[0; -1; 1; -2] 0%

[1] 0%

[1; 1; 0; 1] 0%

[false; false; true; false] 0%

runtime error 0%

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

What is the value of this expresssion?

```
transform (fun (x:int) -> x > 0)
    [0 ; -1; 1; -2]
```

1. [0; -1; 1; -2]
2. [1]
3. [1; 1; 0; 1]
4. [false; false; true; false]
5. runtime error

ANSWER: 4

The ‘`fold`’ design pattern

a general-purpose recursive function

Refactoring code, again

Is there a pattern in the definition of these two functions?

```
let rec exists (l : bool list) : bool =
   begin match l with
   | [] -> false
   | h :: t -> h || exists t
   end
```

```
let rec acid_length (l : acid list) : int =
   begin match l with
   | [] -> 0
   | h :: t -> 1 + acid_length t
   end
```

Refactoring code, again

Is there a pattern in the definition of these two functions?

```
let rec exists (l : bool list) : bool =
    begin match l with
    | [] -> false
    | h :: t -> h || exists t
    end
```

```
let rec acid_length (l : acid list) : int =
    begin match l with
    | [] -> 0
    | h :: t -> 1 + acid_length t
    end
```

base case:
Simple answer when the list is empty

combine step:
Do something with the head of the list and the result of the recursive call

Can we factor out this pattern using first-class functions?

Preparation

```
let rec exists (l : bool list) : bool =
   begin match l with
   | [] -> false
   | h :: t -> h || exists t
   end
```

```
let rec acid_length (l : acid list) : int =
   begin match l with
   | [] -> 0
   | h :: t -> 1 + acid_length t
   end
```

Preparation: introduce a helper

```
let rec helper (l : bool list) : bool =
   begin match l with
   | [] -> false
   | h :: t -> h || helper t
   end

let exists (l : bool list) = helper l
```

```
let rec helper (l : acid list) : int =
   begin match l with
   | [] -> 0
   | h :: t -> 1 + helper t
   end

let acid_length (l : acid list) = helper l
```

First: introduce a helper function that will (eventually) become the same for both definitions.

Abstracting with respect to Base

```
let rec helper (l : bool list) : bool =
   begin match l with
   | [] -> false
   | h :: t -> h || helper t
   end

let exists (l : bool list) = helper l
```

```
let rec helper (l : acid list) : int =
   begin match l with
   | [] -> 0
   | h :: t -> 1 + helper t
   end

let acid_length (l : acid list) = helper l
```

Abstracting with respect to Base

```
let rec helper (base : bool) (l : bool list) : bool =
   begin match l with
   | [] -> base
   | h :: t -> h || helper base t
   end

let exists (l : bool list) = helper false l
```

```
let rec helper (base : int) (l : acid list) : int =
   begin match l with
   | [] -> base
   | h :: t -> 1 + helper base t
   end

let acid_length (l : acid list) = helper 0 l
```

Abstracting with respect to Combine

```
let rec helper (base : bool) (l : bool list) : bool =
   begin match l with
   | [] -> base
   | h :: t -> h || helper base t
   end

let exists (l : bool list) = helper false l
```

```
let rec helper (base : int) (l : acid list) : int =
   begin match l with
   | [] -> base
   | h :: t -> 1 + helper base t
   end

let acid_length (l : acid list) = helper 0 l
```

Abstracting with respect to Combine

```
let rec helper (base : bool) (l : bool list) : bool =
   begin match l with
   | [] -> base
   | h :: t -> h || helper base t
   end

let exists (l : bool list) = helper false l
```

```
let rec helper (base : int) (l : acid list) : int =
   begin match l with
   | [] -> base
   | h :: t -> 1 + helper base t
   end

let acid_length (l : acid list) = helper 0 l
```

Abstracting with respect to Combine

```
let rec helper (combine : bool -> bool -> bool)
               (base : bool) (l : bool list) : bool =
   begin match l with
   | [] -> base
   | h :: t -> combine h (helper combine base t)
   end

let exists (l : bool list) =
  helper (fun (h:bool) (acc:bool) -> h || acc) false l
```

```
let rec helper (combine : acid -> int -> int)
               (base : int) (l : acid list) : int =
   begin match l with
   | [] -> base
   | h :: t -> combine h (helper combine base t)
   end

let acid_length (l : acid list) =
  helper (fun (h:acid) (acc:int) -> 1 + acc) 0 l
```

What about the types?

```
let rec helper (combine : bool -> bool -> bool)
               (base : bool) (l : bool list) : bool =
   begin match l with
   | [] -> base
   | h :: t -> combine h (helper combine base t)
   end

let exists (l : bool list) =
  helper (fun (h:bool) (acc:bool) -> h || acc) false l
```

```
let rec helper (combine : acid -> int -> int)
               (base : int) (l : acid list) : int =
   begin match l with
   | [] -> base
   | h :: t -> combine h (helper combine base t)
   end

let acid_length (l : acid list) =
  helper (fun (h:acid) (acc:int) -> 1 + acc) 0 l
```

What about the types?

```
let rec helper (combine : bool -> bool -> bool)
               (base : bool) (l : bool list) : bool =
   begin match l with
   | [] -> base
   | h :: t -> combine h (helper combine base t)
   end

let exists (l : bool list) =
  helper (fun (h:bool) (acc:bool) -> h || acc) false l
```

```
let rec helper (combine : acid -> int -> int)
               (base : int) (l : acid list) : int =
   begin match l with
   | [] -> base
   | h :: t -> combine h (helper combine base t)
   end

let acid_length (l : acid list) =
  helper (fun (h:acid) (acc:int) -> 1 + acc) 0 l
```

Making the Helper Generic

```
let rec helper (combine : 'a -> 'b -> 'b)
               (base : 'b) (l : 'a list) : 'b =
   begin match l with
   | [] -> base
   | h :: t -> combine h (helper combine base t)
   end

let exists (l : bool list) =
  helper (fun (h:bool) (acc:bool) -> h || acc) false l
```

```
let rec helper (combine : 'a -> 'b -> 'b)
               (base : 'b) (l : 'a list) : 'b =
   begin match l with
   | [] -> base
   | h :: t -> combine h (helper combine base t)
   end

let acid_length (l : acid list) =
  helper (fun (h:acid) (acc:int) -> 1 + acc) 0 l
```

The helpers now have the *same* type.

But they are instantiated differently for the two uses.

List Fold

```
let rec fold (combine: 'a -> 'b -> 'b)
             (base:'b) (l : 'a list) : 'b =
   begin match l with
   | [] -> base
   | x :: t -> combine x (fold combine base t)
   end

let exists (l : bool list) : bool =
    fold (fun (h:bool) (acc:bool) -> h || acc) false l

let acid_length (l : acid list) : int =
    fold (fun (h:acid) (acc:int) -> 1 + acc) 0 l
```

Just rename "helper" to "fold".

`fold` (a.k.a. "reduce")

- Like `transform`, foundational function for programming with lists
- Captures the pattern of *recursion over lists*
- Part of OCaml standard library (`List.fold_right`)
- Similar operations for other recursive datatypes (`fold_tree`)

Using List Fold

```
let rec fold (combine: 'a -> 'b -> 'b)
             (base:'b) (l : 'a list) : 'b =
   begin match l with
   | [] -> base
   | x :: t -> combine x (fold combine base t)
   end

let exists (l : bool list) : bool =
    fold (fun (h:bool) (acc:bool) -> h || acc) false l
```

fold:
general-purpose recursion

combine function:
computes the result given h the head of the list and acc the "*accumulated*" answer given by recursion

base case:
answer when the list is empty

9: Rewrite using fold

0

1
0%

2
0%

3
0%

4
0%

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

How would you rewrite this function

```
let rec sum (l : int list) : int =
   begin match l with
   | [] -> 0
   | h :: t -> h + sum t
   end
```

using fold? What should be the arguments for base and combine?

1. combine is: `(fun (h:int) (acc:int) -> acc + 1)`
   base is: `0`

2. combine is: `(fun (h:int) (acc:int) -> h + acc)`
   base is: `0`

3. combine is: `(fun (h:int) (acc:int) -> h + acc)`
   base is: `1`

4. sum can't be written with `fold`.

Answer: 2

9: Rewrite using fold

0

1
0%

2
0%

3
0%

4
0%

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

How would you rewrite this function

```
let rec reverse (l : int list) : int list =
   begin match l with
   | [] -> []
   | h :: t -> reverse t @ [h]
   end
```

using fold? What should be the arguments for base and combine?

1. combine is: `(fun (h:int) (acc:int list) -> h :: acc)`
   base is: `0`

2. combine is: `(fun (h:int) (acc:int list) -> acc @ [h])`
   base is: `0`

3. combine is: `(fun (h:int) (acc:int list) -> acc @ [h])`
   base is: `[]`

4. reverse can't be written by with `fold`.

Answer: 3

Functions as Values

- We've seen many ways in which functions can be treated as values in OCaml
- Everyday programming practice (in many languages, not just OCaml!) offers many more examples
  - objects bundle "functions" (a.k.a. methods) with data
  - iterators ("cursors" for walking over data structures)
  - event listeners (in GUIs)
  - etc.
- Also heavily used for large-scale computing: Google's MapReduce
  - Framework for transforming (mapping) sets of key-value pairs
  - Then "reducing" the results per key of the map
  - Easily distributed to 10,000 machines to execute in parallel!